

Article

XBoot: A RAPID and Instructional Low-Code Generator for Spring Boot Applications

Basem Y. Alkazemi *  and Mohamed K. Nour 

Department of Software Engineering, College of Computing, Umm Al-Qura University,
Makkah 21955, Saudi Arabia; mknour@uqu.edu.sa

* Correspondence: bykazemi@uqu.edu.sa

Abstract

Developing secure and well-structured web applications using Spring Boot presents a significant challenge, as it requires developers to manage multiple layers, employ framework-specific annotations, and ensure authentication, authorization, and compliance with architectural standards. These complexities often lead to errors among students and novice developers. Although current low-code platforms reduce coding effort, they frequently compromise clarity, modularity, and maintainability. This paper introduces XBoot, a lightweight framework that utilizes a straightforward XML-based domain-specific language (DSL) to automatically generate modular and secure Spring Boot applications. By providing concise specifications for entities, services, routes, and user roles, XBoot generates database entities, service classes, controllers, user interface templates, and integrated security rules. Validation rules are directly enforced from the DSL, and built-in Swagger documentation facilitates interactive API testing. The evaluation was conducted in two phases. Initially, XBoot was validated by generating applications for student-course and flight-booking domains, where less than 50 lines of DSL resulted in 950–1350 lines of Java and HTML code, complete with security and documentation. Subsequently, 10 undergraduate students utilized XBoot in practice. All participants successfully generated and deployed applications within 2–20 min (average ≈ 7), compared to 45–120 min for manual implementation. On a 5-point Likert scale, students rated the reinforcement of layered architecture at an average of 4.0. These findings suggest that XBoot effectively eliminates common structural and security errors, reduces boilerplate complexity through concise DSL specifications, and maintains modularity and transparency—limitations often observed in traditional coding and other low-code platforms.

Keywords: spring boot; domain-specific language (DSL); code generation; web application scaffolding; RBAC; layered architecture; model-driven development; educational tool; component-connector architecture



Academic Editor: Luis Javier
García Villalba

Received: 13 August 2025
Revised: 25 September 2025
Accepted: 28 September 2025
Published: 30 September 2025

Citation: Alkazemi, B.Y.; Nour, M.K.
XBoot: A RAPID and Instructional
Low-Code Generator for Spring Boot
Applications. *Appl. Sci.* **2025**, *15*, 10621.
[https://doi.org/10.3390/
app151910621](https://doi.org/10.3390/app151910621)

Copyright: © 2025 by the authors.
Licensee MDPI, Basel, Switzerland.
This article is an open access article
distributed under the terms and
conditions of the Creative Commons
Attribution (CC BY) license
([https://creativecommons.org/
licenses/by/4.0/](https://creativecommons.org/licenses/by/4.0/)).

1. Introduction

The development of modern web applications increasingly demands systems that are modular, secure, and rapidly deployable. In both enterprise and cloud-native settings, developers must adhere to layered architectures, enforce fine-grained access control, and maintain adaptable codebases as requirements evolve. Meeting these challenges requires frameworks that accelerate backend construction without compromising software engineering principles. Among these, Spring Boot [1] is widely adopted, offering strong support for RESTful APIs, persistence management, modular design, and integrated security.

A defining strength of Spring Boot is its annotation-driven configuration. Annotations such as `@RestController`, `@Service`, `@Repository`, and `@PreAuthorize` eliminate boilerplate and enable declarative, maintainable design. While experienced developers benefit from this abstraction, students and novices often struggle to understand how layers interact, how dependency injection operates, and how security is enforced. Misuse of annotations, inconsistent routing, and blurred separation of concerns are common pitfalls.

Instructional experience confirms that even moderately complex applications, such as those with two entities, JPA repositories, and multiple controllers with Thymeleaf templates, pose challenges for undergraduates. Errors typically arise from misconfigurations, controller–view mismatches, or weak architectural separation, highlighting that these are framework-specific difficulties rather than general programming issues.

Effective strategies in this context emphasize learning through complete, working examples. Studying full applications helps students see how components integrate, while formalizing designs with a domain-specific language (DSL) [2] further strengthens architectural reasoning. By expressing systems declaratively, learners think in terms of structure and behavior rather than low-level details. This promotes clarity, reinforces component boundaries, and supports the transition from coding to deliberate, model-driven design.

The need for accessible backend scaffolding also extends to professional settings, particularly for rapid prototyping and early-stage system design. Existing low-code and scaffolding tools promise speed but often lack transparency, security modeling, and adherence to architectural principles, limiting their educational and professional value [3–5].

To address these gaps, this paper introduces XBoot, a lightweight DSL-based framework for generating modular and secure Spring Boot applications. Using compact XML specifications, developers can define entities, validation rules, roles, routes, and component relationships. XBoot generates the full backend stack: JPA entities, services, REST and view controllers, Thymeleaf templates, role-based security, and Swagger API documentation. XML is chosen for its readability, structure, and wide tool support [6,7]. The role of DSLs in promoting maintainability and modularity is well established in software engineering literature [8,9].

This paper makes the following contributions:

1. Introduces XBoot, a DSL-based framework that generates secure, modular Spring Boot applications from high-level XML specifications.
2. Demonstrates how XBoot improves educational outcomes by enabling learners to practice software architecture and Spring Boot development using traceable, fully functional prototypes.
3. Illustrates how XBoot supports rapid prototyping while preserving architectural traceability and component clarity.
4. Defines and implements a declarative security model using DSL constructs to consistently enforce access control across application routes and services.

Unlike existing tools, XBoot uniquely combines architectural discipline, integrated security, and educational clarity. Every DSL element maps directly to generated code, ensuring traceability and aligning with the component-connector architectural style [10], which emphasizes modular composition and secure coordination.

The paper is organized as follows: Section 2 reviews related work. Section 3 describes the XML DSL engineering process, and Section 4 outlines XBoot’s architecture. Section 5 presents syntax and workflow, Section 6 explains the security model, and Section 7 reports case studies and evaluation. Section 8 discusses educational benefits, Section 9 contrasts XBoot with AI tools, Section 10 compares it with manual development, Section 11 concludes, and Section 12 summarizes limitations.

2. Related Work

The growing emphasis on modularity, scalability, and secure-by-design principles has shaped the evolution of back-end and full-stack application development. Microservice-based architecture, promoted by Fowler and Lewis [11] then expanded by Newman [12] and critically analyzed by Dragoni et al. [13], enables systems to be composed of independently deployable services that align with organizational agility. Likewise, the secure-by-design approach [14] emphasizes the embedding of security principles throughout the system lifecycle, from modeling to implementation. These shifts have catalyzed the development of a wave of tools designed to support backend automation and reduce architectural complexity.

Numerous tools and frameworks have emerged to accelerate backend and full-stack web application development by automating the generation of boilerplate code, data access layers, and user interfaces. These tools offer varying levels of abstraction, automation, and adherence to architectural principles. To evaluate the suitability of existing tools for secure and maintainable backend development, we adopted the following six practical criteria.

- **DSL-Based Modeling:** Whether the tool provides a domain-specific language to define the application structure declaratively. DSLs offer several benefits, including design-to-code traceability, architectural consistency, and modular thinking, which are essential in both professional and educational contexts [2,6]
- **View Generation:** The extent to which the tool supports the generation of user interfaces (none, partial, or full). This capability facilitates rapid prototyping and prepares applications for automated testing using tools such as JMeter [15] and Selenium [16].
- **REST API Support:** Whether the tool scaffolds REST endpoints for CRUD operations and custom business logic. Robust API generation is essential for building functional and integrable web applications [4,17].
- **Security Model Integration:** Support for defining and enforcing access control policies, such as role-based authorization. Given the prevalence of security concerns in web systems, integrated security modeling is a critical aspect of evaluation.
- **Scope:** The range of functionality covered, whether the tool supports backend-only generation, full-stack development, or basic scaffolding.
- **Complexity:** The learning curve, ease of customization, and usability of the tool, particularly in instructional environments. Simplicity and clarity are especially valuable for novice learners, whereas excessive complexity may hinder its adoption in academic contexts.

This paper presents a curated selection of tools, evaluated through a combination of practical experimentation and literature review. The selection process was guided by several criteria, including their popularity in both academic and industrial contexts, accessibility for evaluation, and relevance to back-end and full-stack code generation. Priority was given to solutions that are widely adopted, frequently cited in related research, and representative of well-established low-code development practices. Furthermore, the aim was to encompass a range of modeling paradigms, from DSLs to visual scaffolding environments. Tools that were not publicly accessible, lacked sufficient documentation, or had a narrow functional scope were excluded. Freely available tools were assessed by generating an application similar to that presented in the case study. Commercial tools without free trials were evaluated based on official documents and published academic or industry reviews. The subsequent discussion surveys candidate tools and frameworks that exemplify these trends.

Commercial low-code platforms, such as Mendix [18] offer visual modeling for database design, business logic, and user interfaces, and have even been utilized in educational settings [19]. However, these platforms obscure backend details, lack DSL-based modeling, and are proprietary, which limits their extensibility and transparency. Boo-

tify [20] also facilitates project setup through a wizard-driven interface, but its free version excludes REST API generation, dynamic views such as Thymeleaf, and integrated security. Spring Initializr [21] is confined to project bootstrapping, producing only a build file and directory structure. Spring Roo [22] previously reduced boilerplate through annotation-driven scaffolding, but it is now outdated, reliant on AspectJ, and more challenging to trace, diminishing its educational value. OpenXava [23] generates enterprise applications from annotated domain classes, but its dependence on proprietary annotations results in lock-in and conceals architectural layers crucial for learning.

In addition to these commercial and wizard-driven platforms, a parallel line of DSL-oriented tools has sought to provide higher levels of automation and model-driven expressiveness. DSL-oriented tools offer greater automation but remain limited. CRUDyLeaf [24] is a lightweight DSL-based generator that creates CRUD APIs for entity management, yet it does not generate views or security features, leaving applications incomplete and limiting its educational utility. JHipster [25] integrates Spring Boot with modern frontends and supports advanced security and microservices. While potent, its complexity often overwhelms novices, and empirical studies have highlighted the intricacy of its build and deployment processes [26], raising concerns about maintainability. Educationally, its steep learning curve and production-grade tooling render it more suitable for advanced or DevOps-oriented courses than introductory web engineering. Furthermore, although JDL facilitates domain modeling, JHipster provides limited backend view-layer generation and lacks explicit modeling of secure connectors, reducing its pedagogical value for teaching architectural layering and security design. To consolidate these observations and enable a structured comparison, the surveyed tools are summarized in Table 1 across six key evaluation criteria: DSL support, view generation, REST APIs, security, scope, and complexity.

Table 1. Summary of Related Work.

Tool	DSL-Based	View	REST API	Security Model	Scope	Complexity
Mendix	✗	✓	✓	✗	Full-stack	High
CRUDyLeaf	✓	✗	✓	✗	Backend (CRUD only)	Low
Bootify.io	✗	Partial	✗	✗	Backend starter	Medium
Spring Initializr	✗	✗	✗	✗	Skeleton project only	Low
JHipster	Partial	✓	✓	✓	Full-stack	High
Spring Roo	Partial	Partial	✗	✗	Monolithic Spring apps	Medium
OpenXava	✗	✓	✗	✗	Full-stack	Low

Note: ✓ = Full support, ✗ = No support, Partial = Limited or component-specific support.

DSL research efforts provide another perspective on the role of DSLs in low-code and no-code (LC/NC) development. Slifka and Pergl [27] applied Normalized Systems Theory to functional DSLs, emphasizing modularity and sustainability rather than full-stack generation. Martínez-Lasaca et al. [28] introduced Dandelion, a graphical DSL workbench for industrial LC/NC development, but its infrastructure requirements make it less lightweight. Chavarriaga et al. [29] proposed a JSON-based DSL for web applications, offering concise notation but weaker validation and schema enforcement compared to XML. Pinho et al. [30] reviewed usability in LC/NC platforms, stressing the importance of developer experience and accessibility. Dalibor et al. [31] investigated DSL-driven customization of LC/NC platforms for digital twins, demonstrating adaptability but reinforcing the domain-specific nature of many DSL efforts.

Collectively, these studies elucidate both the potential and the constraints of contemporary DSL and low-code/no-code methodologies, thereby establishing the foundation for

our comprehensive assessment of existing frameworks. The review of existing frameworks shows that while many tools simplify development, they often lack integrated security, view generation, and transparent design-to-code traceability. In parallel, recent DSL research underscores the central role of DSLs in advancing low-code and no-code platforms, but these efforts are either domain-specific or limited in scope, making them less applicable to lightweight educational contexts. Together, these observations highlight the need for a general-purpose DSL-driven framework that combines the strengths of low-code tools with the rigor and transparency of DSLs.

To address the recurring gaps observed in existing tools and DSL research, the design of XBoot was guided by the following requirements:

RQ1: Enable full-stack generation from a single XML DSL specification.

RQ2: Provide built-in security with role-based access control.

RQ3: Support generation of both REST APIs and server-rendered Thymeleaf views.

RQ4: Ensure transparent, traceable Spring Boot code aligned with best practices.

The development of the XML-based DSL, which forms the foundation of XBoot, was directly informed by these requirements, as will be discussed in the following section.

3. Engineering Process of the XML-Based DSL

The XML-based DSL at the core of XBoot was engineered following established practices in DSL and model-driven engineering [2]. The process comprised four stages: domain modeling, XML syntax design, schema formalization, and processor implementation. This provided a disciplined path from requirements analysis to the implementation of a working tool, ensuring that the language was both technically sound and pedagogically suitable. In the paper, this structured process provides the foundation for the subsequent Sections 4 and 5, which elaborate the architectural perspective of XBoot and detail the DSL design and code generation process.

3.1. Domain Modeling

The process began with domain modeling, where the essential concepts of Spring Boot applications were identified. These included entities and their fields, relationships between entities, repositories for persistence, services for business logic, controllers for interaction handling, views for user interface pages, and routes for navigation and interaction between components. Security requirements were also considered, with roles representing access control responsibilities.

This stage also incorporated educational requirements, as XBoot is intended for both professional prototyping and instructional use. Readability, simplicity, and verifiability were prioritized, ensuring that learners can quickly understand how domain concepts map to generated artifacts, while instructors can validate student specifications against clear structural rules.

The result of this stage was a domain model capturing the recurring concepts and relationships of a typical Spring Boot application, enriched with educational considerations.

3.2. XML Syntax Design

Once the domain model was established, the next step was to define the DSL's concrete syntax in XML. Each domain concept was mapped to an XML element or attribute. For example:

<Entity> elements capture domain objects.

<Field> elements define entity attributes, with attributes such as name and type.

<Relation> elements represent associations (e.g., one-to-many, many-to-many).

<Role> elements specify access control roles.

<Route> elements describe navigational paths and associated views.

This mapping establishes the link between abstract syntax (concepts) and concrete syntax (notations). XML was chosen because of its familiarity, machine-readability, and support in common development tools. It provides a structured format that can be easily parsed and validated, while remaining accessible to learners.

3.3. XSD Formalization

To ensure correctness and enforce constraints, an XML Schema Definition (XSD) was created. The schema defines:

- Which elements are allowed (e.g., `<Entity>`, `<Field>`, `<Relation>`, `<Role>`, `<Route>`).
- Cardinalities (e.g., an `<Entity>` must have one or more `<Field>` elements).
- Attribute constraints (e.g., type in `<Field>` must match a predefined set of data types).
- Nesting rules (e.g., `<Relation>` must reference valid `<Entity>` names).

Schema-based validation [32] ensures that any DSL specification is structurally correct before processing. This reduces runtime errors in code generation and provides users with immediate feedback if their model is incomplete or inconsistent. The use of XSD also improves verifiability and supports tool-assisted editing in IDEs that understand XML schemas.

3.4. Processor Implementation

The final stage was the implementation of a processor that parses valid XML specifications and transforms them into Spring Boot code. A Java-based parser interprets the XML input, builds an internal representation of the model, and passes it to modular code generators. Each generator corresponds to one application layer, some of which are:

- `EntityGenerator`: creates annotated Java entity classes.
- `RepositoryGenerator`: produces Spring Data JPA repositories.
- `ServiceGenerator`: generates service classes with CRUD and domain logic.
- `ControllerGenerator`: builds REST and MVC controllers.
- `ViewGenerator`: generates Thymeleaf templates for server-side rendering.
- `SecurityGenerator`: configures role-based access control.

These generators apply model-to-text transformation patterns [2], producing traceable, standard Spring Boot artifacts. The modular design supports separation of concerns [10], as each generator is independent and can evolve without affecting others. This also ensures extensibility, allowing future generators (e.g., for APIs, messaging, or external integrations) to be added incrementally.

The processor thus serves as the bridge between DSL specification and executable Spring Boot code, completing the engineering lifecycle.

4. Architectural Overview of XBoot

XBoot is a lightweight modular framework designed to generate secure Spring Boot application prototypes using a model-driven approach. In contrast to black-box low-code tools, XBoot prioritizes architectural transparency, traceability, and alignment with the best practices in layered backend development. The framework transforms a compact XML-based specification into a fully runnable Spring Boot application through a structured toolchain comprising the following core components:

1. **XML-based DSL:** Developers specify the application architecture using a structured DSL. This includes the definitions of entities, fields, controllers, routes, validation constraints, access roles, and view mappings.
2. **XSD Schema Validator:** XML is validated against a dedicated XSD schema to enforce structural correctness, tag constraints, and data type compatibility. This ensures real-time error detection and improves the IDE integration.

3. Java-based DSL Parser: A custom parser maps DSL elements (e.g., <Entity>, <Field>, <Validation>, <Route>, <Connector>) to an internal object model. This model preserves entity relationships, validation rules, and security mappings to ensure accurate code generation.
4. Spring Boot Code Generator: Based on the internal model, XBoot generates a complete executable backend project that includes the following:
 - JPA entity classes with validation annotations.
 - Repositories extending JpaRepository.
 - Service classes encapsulating business logic.
 - REST controllers with route and role-based access control.
 - View controllers for form rendering and data listing.
 - Thymeleaf templates with Bootstrap 5 styling and validation error rendering.
 - Spring Security configuration for enforcing role-based access.
 - Swagger configuration and annotations (@Api, @ApiOperation) for documenting and testing REST APIs.
 - Project metadata (e.g., pom.xml, application.properties, and MainApp entry point).

Collectively, these generated artifacts constitute an integrated toolchain encompassing all backend layers, as depicted in Figure 1. This toolchain not only visualizes the flow of transformations but also illustrates how the design requirements (RQ1–RQ4) are operationalized across components.

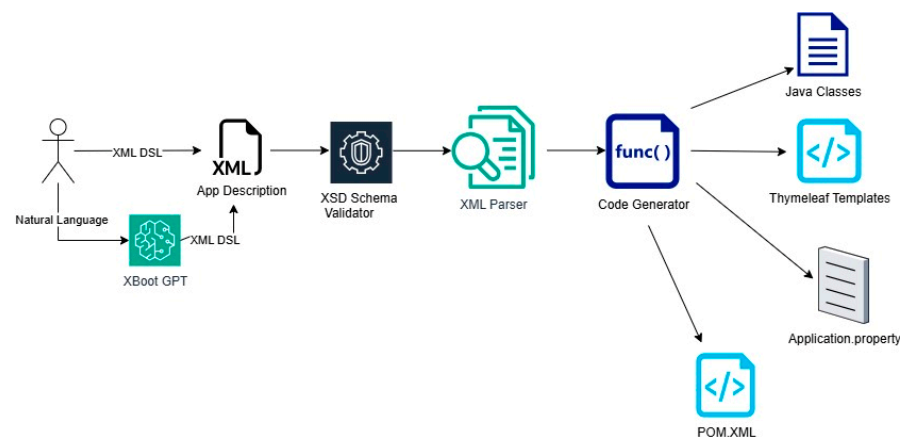


Figure 1. XBoot toolchain for generating Spring Boot applications from DSL specifications.

The alignment between the research questions and the architectural components underscores the systematic design of XBoot. Specifically, the XML DSL, Validator, and Parser together address RQ1 by facilitating unified full-stack generation. The Parser and Code Generator address RQ2 by maintaining role definitions and producing secure controllers with Spring Security. Furthermore, the Code Generator's outputs for REST controllers and Thymeleaf templates correspond to RQ3. Finally, the adoption of standard Spring Boot conventions in entities, services, and configuration files ensures RQ4. These mappings are detailed in Table 2.

Table 2. Mapping of research questions (RQ1–RQ4) to XBoot architectural components.

Research Question	Requirement	Architectural Support in XBoot
RQ1	Full-stack generation from a single XML DSL	XML DSL block provides the model; XSD Schema Validator ensures correctness; XML Parser transforms into an internal model; Code Generator produces Java classes, Thymeleaf templates, configuration files
RQ2	Built-in security with role-based access control	XML Parser preserves role definitions; Code Generator outputs secure REST controllers, Spring Security configuration, and supporting metadata (e.g., pom.xml)
RQ3	Generation of both REST APIs and Thymeleaf views	Code Generator produces REST controllers with Swagger annotations and Thymeleaf templates for forms and listings
RQ4	Transparent, traceable Spring Boot code	Code Generator outputs JPA entities, repositories, services, and configuration files (application.properties, pom.xml) using standard Spring Boot conventions

As delineated in Table 2, the architectural components of XBoot collectively address the identified research questions, thereby ensuring comprehensive full-stack generation, integrated security, view support, and traceable code. While this systematic mapping underscores the rigor of XBoot’s design, effective utilization of the DSL necessitates support for students and novice developers who may encounter difficulties with XML syntax and modeling conventions. To address this challenge and further enhance usability, XBoot is supplemented by a custom-trained GPT-based assistant, XBoot GPT [33]. This language model is fine-tuned to understand backend development concepts and can translate informal system descriptions into valid XBoot DSL code. Students and developers can interact with XBoot GPT to rapidly bootstrap their models, reducing time spent on DSL syntax and structure. Importantly, the generated XML is still validated and transformed by the core XBoot generator, ensuring traceability, determinism, and architectural consistency.

5. DSL Design and Code Generation Process

The design of XBoot is guided by the principles of modularity, traceability, and security-by-design. These principles are implemented using a structured XML-based DSL and a layered code generation pipeline. Unlike traditional low-code environments, XBoot leverages textual DSL to promote structural clarity and customization. This approach aligns with emerging frameworks that propose DSL extensibility as a pathway to complete no-code solutions [34]. This section outlines the core elements of the DSL and how they drive the automated creation of Spring Boot applications.

XBoot’s DSL supports both structural (e.g., entities, fields, relationships) and behavioral modeling (e.g., controllers, routes, roles). Developers use a single XML file to describe the entire application architecture, which reduces repetitive code and improves architectural consistency across all layers.

5.1. DSL Structure and Modeling Semantics

At the heart of XBoot is a structured XML DSL that defines the static and dynamic aspects of the application. Tags such as <Entity>, <Field>, <Controller>, <Connector>, <Route>, and <Validation> are used to capture the domain models, access roles, API paths, and field-level constraints.

Each XML specification was validated against an XSD schema to ensure well-formedness and design correctness. This schema-aware modeling approach supports both auto-completion and early error detection in modern integrated IDEs. Table 3 outlines the main DSL tags and attributes allowing for fine-grained input constraint modeling at the field level.

Table 3. XBoot DSL Tags and Attributes.

Tag	Attribute	Allowed Values/Format	Description
<Application>	name	String (e.g., StudentCourses)	Defines the application name and serves as the root element of the XML document.
<Entity>	name	String	It declares a persistent domain class.
<Field>	name	String	Name of entity field.
	type	String, Integer, Date, etc.	Data type of the field.
	required	true, false	Indicates whether the field is mandatory.
	relation	OneToMany, ManyToOne, ManyToMany	Defines the relationship with another entity.
<Validation>	type	NotBlank, NotNull, Size, Pattern, Email	Type of validation constraints.
	min, max	Integer	Used with Size to define the length bounds.
	regex	Java-compatible regex (e.g., \d{10})	The pattern is used to enforce the input format.
<Service>	name	String (optional)	Name of the service class (automatically generated if omitted).
<Controller>	name	String	Name of the controller.
	for	Matches <Entity> name	This indicates which entity the controller handles.
<Repository>	name	String (optional)	Name of the repository interface (auto-generated if omitted).
<Connector>	from, to	Names of controller and entity	Secured interaction between components.
	role	ADMIN, USER, etc., (comma-separated)	Roles authorized to use connectors.
	type	REST, MVC, async, secured	Communication style and access type
<View>	for	Name of entity or controller	Indicates the subject for which the view was generated.
	template	e.g., form.html, list.html	Thymeleaf HTML template to render.
<Route>	path	e.g., /students/register	Defines the URL path.
	view	View name (must match template)	View to render this route.
	method	GET, POST, PUT, DELETE	HTTP method to handle.
	role	USER, ADMIN, etc.	Role required to access the route.
	controller	Name of controller	The route to the backend controller is mapped.

This DSL supports both functional unit modeling (entities, repositories, and services) and interaction modeling (routes and access roles). For example, the following snippet in Box 1 defines an enrollment entity with two relationships.

Box 1. XML-DSL Definition of the Enrollment Bridge Table.

```

<Entity name="Enrollment">
  <Field name="student" type="Student" relation="ManyToOne"/>
  <Validation type="NotBlank"/>
  <Validation type="Size" min="3" max="10" message="Student name must be between 3 and 10 characters"/>
  <Field name="course" type="Course" relation="ManyToOne"/>
  <Field name="grade" type="String"/>
</Entity>

```

The simplicity of this definition belies the complexity of the backend components it generates: JPA annotations, repository interfaces, controller endpoints, view templates, and security layers.

For standardization and future extensibility, we formally defined the core syntax of the XBoot DSL using the Extended Backus-Naur Form (EBNF) [32], as outlined in Appendix A.2. This grammar captures both functional unit modeling, including entities, repositories, and services, and interaction modeling, such as routes and role-based access definitions. This facilitates precise tooling, such as syntax validation and DSL-aware editors, enables schema evolution tracking through formal diffs, and supports integration with parser generators and language workbenches. Moreover, clearly defined grammar enhances the future portability of DSL and its adoption by third-party tools or educational environments, where consistent interpretation of models is critical.

5.2. Security and Role-Based Connectors

XBoot treats security as a first-class concern by embedding role-based access control (RBAC) directly into the system architecture. The <Connector> and <Route> elements support a role attribute that declaratively defines the user roles that can access a given path

or service. These access rules are automatically translated into Spring Security annotations (e.g., @PreAuthorize) or configuration entries to ensure consistency across the codebase.

This architectural embedding of access control aligns with the concept of secure connectors [35], where interactions between components are governed by structure and verifiable security constraints. For instance, Box 2 elucidates the interface connecting the controller and service layers of the application:

Box 2. Connector Definition between Controller and Service layers.

```
<Connector from="EnrollmentController" to="EnrollmentService" role="AD-
MIN,MANAGER"/>
<Route path="/enrollments" view="list" role="USER" controller="Enroll-
mentController"/>
```

This model eliminates the need to manually define and synchronize access control policies in scattered controllers or configuration files. It also enables traceable changes; when a role is updated in the DSL, all related access points are regenerated consistently.

5.3. Code Generation and Traceability

XBoot enforces model-to-code traceability by treating DSL as a single source of truth. Every entity, field, route, and validation rule defined in the XML specification is faithfully reflected in the generated backend code, thereby ensuring consistency across all layers of the application. For instance, the DSL snippet presented in Box 3 specifies a title field within the course entity, accompanied by a validation constraint.:

Box 3. DSL Definition of Validation on the Title Element.

```
<Entity name="Course">
  <Field name="title" type="String" required="true">
    <Validation type="NotBlank"/>
    <Validation type="Size" min="3" max="100" message="Title must be be-
tween 3 and 100 characters"/>
  </Field>
  <Field name="description" type="String"/>
</Entity>
```

From this DSL fragment, XBoot generates various classes across different layers. Some examples are:

- JPA Entity (Course.java) with its validations is shown in Box 4.

Box 4. Generated JPA Entity (Course.java) with Validations.

```
@NotBlank
@Size(min = 3, max = 100, message = "Title must be between 3 and 100 char-
acters")
private String title;
private String description;
```

- Thymeleaf form template (course-form.html) is shown in Box 5.:

Box 5. Generated Thymeleaf Template with Validation.

```
<input type="text" th:field="*{title}" class="form-control"/>
<div th:if="{#fields.hasErrors('title')}" th:errors="*{title}"
```

- REST controller for course management (CourseController.java) is given in Box 6.

Box 6. Generated REST Controller for Course Management.

```
@Api(tags = "Course")
@RestController
@RequestMapping("/courses")
public class CourseController {

    @ApiOperation("Create a new course")
    @PostMapping
    public ResponseEntity<Course> saveCourse(@Valid @RequestBody Course
course) {
        return ResponseEntity.ok(courseService.saveCourse(course));
    }
}
```

This example demonstrates how a single field definition in the DSL propagates across multiple architectural layers:

- To the data model: The DSL specification is translated into Java code within the entity class, where validation constraints (such as `@NotBlank` and `@Size`) are directly applied to fields. These annotations ensure that invalid data are rejected early at the persistence or controller layer, preserving data integrity and reducing the need for manual checks.
- To the view layer: The corresponding Thymeleaf form is automatically generated with `th:field` bindings linked to the entity fields. Validation messages are displayed in context using `th:errors`, providing immediate and localized feedback to the user. This ensures that the client-side and server-side validations are aligned and easy to maintain.
- To the REST API: The REST controller generated by XBoot includes the appropriate method for creating or updating the entity. Swagger annotations (`@Api`, `@ApiOperation`) document the endpoint clearly, while `@Valid` ensures that any request payload adheres to the validation rules defined in the DSL. This leads to consistent runtime behavior and transparent API contracts for clients.

Together, these mappings reflect the core benefit of XBoot's design: complete traceability from the design to the implementation. When a developer updates the DSL, by adjusting a constraint, renaming a field, or adding a new entity, those changes are automatically reflected across all affected layers. This eliminates boilerplate, prevents configuration drift, and reinforces architectural understanding. In instructional settings, this transparency helps learners observe how a high-level model drives consistent changes across the back-end, reinforcing their understanding of the separation of concerns, validation propagation, and layered architecture, thereby making XBoot an effective teaching and prototyping tool.

To further clarify how XBoot enforces traceability across layers, Figure 2 maps key elements of the XML-based DSL to the corresponding backend components generated

by the system. This visual representation demonstrates how a single source of truth translates into concrete application artifacts, such as JPA entities, service classes, REST controllers, Thymeleaf views, and role-based access control policies. By making these mappings explicit, the figure reinforces XBoot's pedagogical value in showing students how high-level specifications propagate across architectural layers. This traceability not only ensures consistency and maintainability but also aids in teaching layered design, validation propagation, and secure software construction.

	JPA Entity	Repository	Service Class	REST Controller	Thymeleaf View	Security Config	Swagger Doc
<Entity>	✓	✓					
<Field>	✓				✓		
<Validation>	✓			✓	✓		
<Controller>			✓	✓			
<Service>			✓				
<Route>				✓		✓	✓
<Connector>						✓	
<View>					✓		
Legend	✓	Contribution present No contribution					

Figure 2. Mapping DSL to Generated Artifacts.

6. Security Modeling and Access Control

Security in multi-user web applications must be designed from the ground up to ensure that users only access data and actions for which they are authorized. XBoot embeds RBAC directly into the architectural model using declarative tags within the DSL. This ensures that security concerns are not treated as an afterthought but are defined alongside the core application logic.

XBoot supports access control through two primary DSL constructs:

- <Route> which governs the view level and endpoint access.
- <Connector> which controls the component-level communication and service access.

Together, these tags allow developers to define fine-grained access policies for both front-end views and back-end services.

6.1. Route-Based Access Control

The <Route> element defines a navigational path within the application and specifies the user roles allowed to access it. Each route entry includes the following:

- A URL path (path).
- The HTTP method (method).
- The view template (view) rendered.
- The controller handling the route (controller).
- The authorized role(s) (role).

For example, the following DSL snippet in Box 7 defines which user roles have permission to access specific views or REST endpoints.

Box 7. XML-DSL RBAC Definition for Application Users (USER/ADMIN).

```
<Route path="/students" view="student-list" method="GET" controller="StudentController" role="USER"/>
<Route path="/enrollments/new" view="enrollment-form" method="POST" controller="EnrollmentController" role="ADMIN"/>
```

These DSL entries are translated as follows:

- Controller methods are annotated with `@PreAuthorize` to enforce access at runtime.
- Spring Security configuration that maps HTTP paths to the required roles.
- Thymeleaf conditionals for selectively rendering view elements based on the user role.

The code snippet included in the generated view controller code is presented in Box 8.

Box 8. Generated Post Method for the Enrollments Controller.

```
@PreAuthorize("hasRole('ADMIN')")
@PostMapping("/enrollments/new")
public String saveEnrollment(@ModelAttribute Enrollment enrollment) {
    enrollmentService.saveEnrollment(enrollment);
    return "redirect:/enrollments";
}
```

The corresponding Thymeleaf template (enrollment-list.html) shown in Box 9 will include a conditional “Add Enrollment” link generated only for users with the ADMIN role.

Box 9. Generated Thymeleaf Snippet (enrollment-list.html) with Conditional Role-Based Link.

```
<a th:if="${#authorization.expression('hasRole('ADMIN')')}"
th:href="@{/enrollments/new}" class="btn btn-primary">
    Add Enrollment
</a>
```

This layered enforcement ensures that even if a user bypasses the UI, unauthorized POST requests are rejected at the controller level. The consistent translation of DSL-defined roles into both backend and frontend layers reinforces access integrity and reduces the likelihood of security oversights. The corresponding RBAC configuration class generated is listed in Appendix A.3. Backend access control is strictly enforced via Spring Security annotations (e.g., `@PreAuthorize`), ensuring that role-based restrictions are consistently applied across the application.

6.2. Component-Level Access with Connectors

While `<Route>` secures navigation and endpoints, the `<Connector>` tag secures the interactions between components, such as between controllers and domain models. This design is influenced by the secure connector paradigm proposed by Shin et al. [35], where security semantics are embedded into the system’s interaction logic. Each connector includes the following:

- The source component (typically a controller).
- The destination component (usually an entity or service).

- The permitted role(s).
- The connector type (REST, MVC, async, etc.).

An example DSL specification of a connector is shown in Box 10.

Box 10. DSL Snippet Defining a Connector with Role and Type.

```
<Connector from="EnrollmentController" to="Enrollment" role="USER"
type="REST" />
```

This connector declares that the *EnrollmentController* is allowed to access the *Enrollment* entity using RESTful operations on behalf of users with the USER role. The generator translates this into method-level annotations and path restrictions to ensure that only authenticated users with the right role can trigger those interactions. If a controller is linked to an entity through a secure connector, all corresponding controller methods are consistently protected.

6.3. Generated Security Configuration

In the current implementation of XBoot, Spring Security is configured to load user credentials and role assignments from the `application.properties` file. This approach intentionally avoids hardcoding sensitive data within Java classes and enables straightforward externalization of authentication details. As a result, developers and instructors can update usernames, passwords, and roles without touching the Java source code, which streamlines testing and credential rotation in both educational and lightweight production environments.

The XBoot generator produces two key security artifacts:

- A `SecurityConfig.java` class that maps HTTP paths to role-based access rules using standard Spring Security constructs.
- An `application.properties` file that includes pre-encoded credentials and associated role assignments for predefined users.

An excerpt of the generated `SecurityConfig.java` class is shown in Box 11.

Box 11. Generated Security Configuration (`SecurityConfig.java`).

```
http
    .authorizeHttpRequests(auth -> auth
        .requestMatchers("/students").hasRole("USER")
        .requestMatchers("/enrollments/new").hasRole("ADMIN")
        .anyRequest().authenticated()
    )
    .formLogin()
    .httpBasic();
```

An excerpt of the generated `application.properties` file with predefined users is shown in Box 12.

This configuration supports both form-based login for web interfaces and HTTP basic authentication for RESTful access, making it adaptable for classroom environments, API testing tools like Postman, and automated clients. By externalizing credentials in the `application.properties` file, XBoot promotes maintainability and adheres to secure development practices.

Crucially, this setup reflects the DSL-driven model: role attributes defined in <Route> or <Connector> are automatically translated into Spring Security configurations, ensuring consistent enforcement across controllers and views. This declarative, end-to-end approach simplifies application security while providing the transparency and structure necessary for instructional use and real-world development.

Box 12. Generated Application Properties with Predefined Users (application.properties).

```
# Security: In-memory users
app.security.user.username=user
app.security.user.password=password
app.security.user.roles=USER

app.security.admin.username=admin
app.security.admin.password=password
app.security.admin.roles=ADMIN, USER
```

7. Case Study and Evaluation

The evaluation of XBoot was conducted through two complementary modes. First, the authors validated the tool by manually specifying and generating two different applications to ensure correctness and completeness. Second, a preliminary usability study was conducted with 10 undergraduate students who had previously completed formal coursework in Spring Boot and Java programming. The students participated under two conditions: four remotely and six in class under direct supervision. In both settings, XBoot GPT was employed to generate DSL specifications from natural language descriptions. The study was designed to evaluate XBoot’s practical usability, generation accuracy, and educational impact within realistic academic contexts.

7.1. Author’s Evaluation of XBoot Fidelity

The first case study models a university registration system comprising three entities: Student, Course, and Enrollment entity. These were defined in the DSL with attributes, validation rules, and relational constraints (e.g., foreign key references). For example, Enrollment includes a composite structure referencing both Student and Course. The application models three core entities: Student, with fields such as name, email, and mobile; Course, with a title and optional description; and Enrollment, which links students to courses and optionally stores a grade. This mirrors a normalized many-to-many schema, where many students can register for many courses via an intermediary entity that can also store additional contextual data like grades. The relationships are encoded using “ManyToOne” mappings from Enrollment to both Student and Course, which XBoot correctly decomposes into two one-to-many relationships. The DSL also specifies access roles, routes, and component interactions.

A partial specification is shown below in Listing 1 (with the full listing included in Appendix A.4.).

This DSL acts as a single source of truth for the system architecture, encapsulating both the structure (entities, fields, relationships) and behavior (routes, roles, access control). Each DSL element is mapped to specific backend components, as outlined in Section 4. The execution log is provided in Appendix A.1., which shows all the steps and artifacts generated.

Listing 1. XBoot DSL representation for the Student–Course case study.

```

<Application name="StudentCourses">
  <!-- Entities -->
  <Entity name="Student">
    <Field name="name" type="String" required="true">
      <Validation type="NotBlank"/>
    ...
  <!-- Controllers -->
  <Controller name="StudentController" for="Student"/>
  <!-- Routes -->
  <Route path="/home" view="home" method="GET" role="USER"/>
  <Route path="/students" view="student-list" method="GET" controller="StudentController" role="USER"/>
  ...
  <!-- Security Connectors -->
  <Connector from="StudentController" to="Student" role="USER"
type="REST"/>
  <Connector from="CourseController" to="Course" role="USER" type="REST"/>
  " type="REST"/>
  ...
</Application>

```

Passing the DSL to the XBoot toolchain produced a fully functional Spring Boot project with the following outputs, as shown in Table 4:

Table 4. Case Study Generated Artifacts.

Layer	Artifacts Generated
Model Layer	JPA entity classes with field-level validation annotations (@NotBlank, @Size)
Repository	Interfaces extending JpaRepository for each entity
Service Layer	Service classes exposing entity operations (e.g., saveStudent, findAllCourses)
Controller Layer	REST and view controllers, annotated with Swagger and Spring Security
View Layer	7 Thymeleaf templates: 2 per entity (form, list) + 1 homepage (home.html)
Security	SecurityConfig.java, application.properties with roles, routes, login config
Documentation	Swagger UI enabled via @Api, @ApiOperation, and OpenAPI integration

This shows that XBoot successfully transforms a compact DSL into a complete, runnable, secure, and structured web application without manual coding. A total of 7 Thymeleaf templates were generated: six for data insertion and listing (two per entity), and one homepage view (home.html). The generated web application compiled and ran without errors and required no manual edits for handling student entry, listing, and enrollment in various courses' functionality.

Several screenshots of the generated pages are provided below for demonstration purposes. Figure 3 shows the student's registration form at the following route: illustrating validation messages due to missing entries in the email and mobile number.

Figure 3. Generated Student details Entry form.

Once the full details are entered correctly, the details are shown in the student-list.html page at route: as shown in Figure 4.

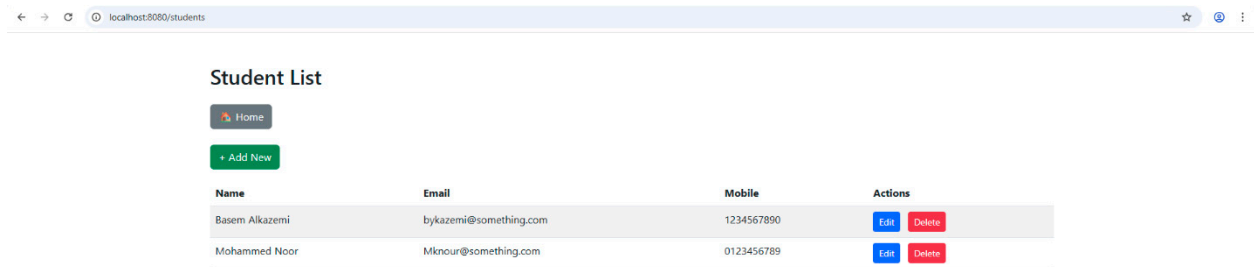


Figure 4. Student listing page with edit and delete capability.

Corresponding pages are also generated for the Course entity (entry form and listing), as shown in Appendix B.1. Since the Student–Course relationship is many-to-many, an Enrollment bridge table is used to capture this association. The Enrollment form (Appendix B.2.) allows users to register students in courses by selecting from drop-down lists of existing entries. The resulting enrollment records are displayed in the enrollment-list.html view (Appendix B.3.), which lists all students enrolled in courses.

The simplicity of the user interface is intentional, as the primary goal of this study is to demonstrate XBoot’s ability to generate fully functional backend applications from a high-level specification. Visual styling and branding are outside the current scope but can be easily incorporated into the Thymeleaf generator in future enhancements.

Figure 5 provides a visual overview of the generated application architecture, illustrating how the DSL model is propagated across backend layers including the entity, service, repository, controller, and view tiers alongside security and API documentation configurations.

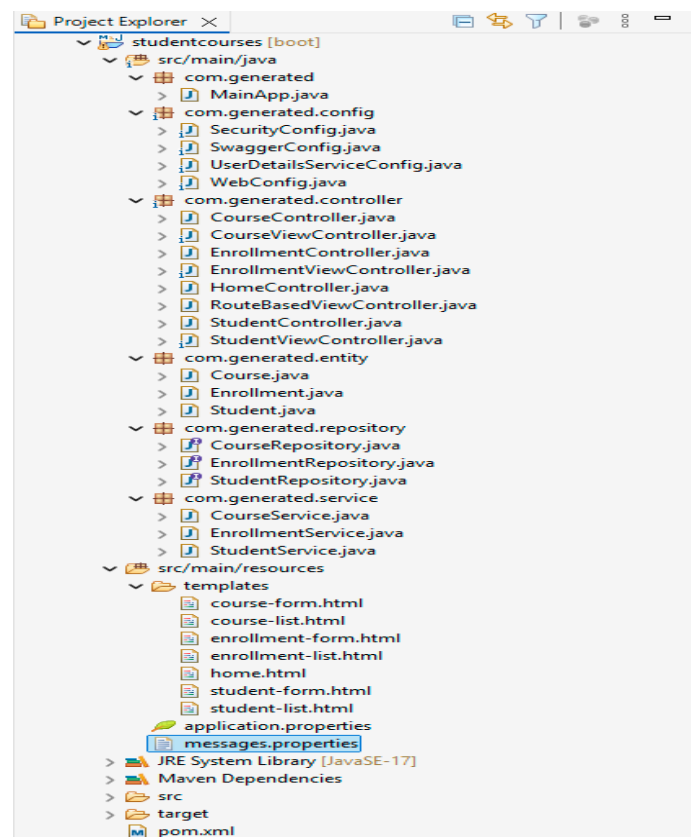


Figure 5. Generated StudentCourse Spring Boot application.

To further test generality, we specified a flight reservation system DSL (given in Appendix A.5.) comprising four entities: Passenger, Airplane, Flight, and Reservation. Each entity includes diverse field types, validation constraints, and inter-entity relationships, e.g., Reservation defines ManyToOne links to Flight, Passenger, and Airplane. The validation rules include patterns, ranges, and email formats.

Despite the domain shift and slight increase in complexity, XBoot generated the complete backend stack from the DSL without requiring changes to the generator. The generated application includes the same set of classes and files as in the previous case study, with differences in counts and details. This confirms XBoot’s capacity to handle larger relational models across varied application domains with consistent generation quality.

The evaluation of the StudentCourses and FlightBooking case studies demonstrated that XBoot reliably transforms compact, declarative specifications into complete, secure, and modular Spring Boot applications. These examples highlight the expressive power of the DSL and the generator’s ability to produce all backend layers (entities, services, repositories, controllers, views, and security configurations) automatically and without manual intervention. The consistency of results across distinct domains reinforces XBoot’s strengths in modular architecture, traceability, and security enforcement. As such, XBoot proves to be a valuable tool for instructional use, model-driven engineering, and rapid prototyping, where reducing boilerplate and preserving architectural consistency are key to success. Table 5 lists the different metrics resulting from the case studies.

Table 5. Summary of Evaluation Metrics.

Metric	StudentCourses	FlightBooking
DSL Entities	3	4
Total Fields	11	22
Controllers (REST + View)	6	8
Services and Repositories	6	8
Thymeleaf Views	7	9
Protected Routes	6	5
Swagger-Documented Endpoints	6	5
Java + HTML Lines of Code	~950	~1350
Time to Build Manually (on average)	~45 min	~60 min
Time Using XBoot (on average) -from typing specification in XBoot GPT to running application	~5 min	~5 min

Manual development times were obtained by both authors independently implementing the full applications by hand; the reported values represent the average of their efforts. In parallel, the same applications were generated using XBoot during the evaluation study, with average end-to-end times (from specification entry in XBoot GPT to a running application) of approximately five minutes.

The prototype was evaluated against the core design goals of XBoot, including modularity, transparency, security integration, educational effectiveness, and prototyping efficiency. The evaluation involved applying XBoot to both case studies and assessing the generated artifacts against each objective. The results of this assessment are summarized in Table 6, which highlights how well XBoot meets its intended design principles.

Table 6. Summary of XBoot Design Goals Evaluation.

Criterion	Evaluation
Modularity	Each concern (entity, service, controller, and view) was generated separately and mapped via DSL, as shown in Figure 2.
Transparency	Every code artifact is traceable to the DSL tag. No hidden logic or dependencies were observed.
Security Integration	Connectors and routes encode roles that are translated into Spring Security annotations.
Educational Utility	The generated code serves as a clean template for teaching standard Spring Boot architecture and security.
Prototyping Fidelity	The generated applications were examined by hand on Eclipse IDE and found to be complete, runnable, and structurally correct.
Complexity Reduction	Approximately 40 XML lines produce >900 lines of code spanning all backend layers including Java code, Thymeleaf HTML, and application properties.

As shown in Table 6, XBoot successfully meets the core design objectives. It promotes modular design by cleanly separating layers and linking them through a compact DSL. All generated code is transparent and traceable to DSL specifications, which supports both debugging and pedagogical exploration. Security is embedded declaratively via routes and connectors, automatically enforced in the generated application. Furthermore, the tool greatly accelerates prototyping while reducing cognitive overhead, especially for learners encountering Spring Boot’s layered architecture for the first time. These attributes make XBoot particularly well-suited for rapid instructional deployment and backend development prototyping. In fact, XBoot can also play a significant role in establishing a software product line (SPL) [36] for web application development and prototyping.

All generated artifacts, including the StudentCourses and FlightBooking applications, can be reproduced using the instructions and DSL examples available in the public GitHub repository (XBoot v1.0, commit hash: 4e8a783; see Data Availability section).

7.2. Preliminary Evaluation with Student Participants

To complement the author-led evaluation, we conducted a preliminary proof-of-concept (POC) study with ten senior undergraduate software engineering students. The goal was not to produce statistically generalizable results, but to verify XBoot’s functionality and obtain provisional insights into its educational value.

Participants, all of whom had prior Spring Boot experience and achieved A/A+ grades in relevant coursework, followed a standardized workflow: (1) describe application requirements in natural language, (2) use XBoot GPT to generate DSL specifications, (3) generate a Spring Boot application with XBoot, (4) run the JAR and verify CRUD operations, Thymeleaf views, and RBAC, and (5) provide feedback through a structured post-experiment survey. The survey (Appendix A.7.) included nine questions covering usability, correctness, and learning impact. This structured process ensured consistent engagement with XBoot across all participants and allowed us to capture both quantitative and qualitative insights into its usability and educational value. The responses, summarized below, provide an initial perspective on how effectively XBoot supports rapid application generation while reinforcing key Spring Boot concepts.

7.2.1. Results

Table 7 summarizes student feedback across both case studies. Most participants reported completing application generation and deployment within minutes, with the majority requiring less than ten. By comparison, manual development of similar case studies in prior Web Engineering lab assignments typically required 90–120 min, where students were tasked with implementing applications to consolidate their understanding of course concepts. This contrast underscores the substantial acceleration achieved with XBoot.

Table 7. Student Feedback Summary on the Two Case Studies.

Questions	Summary of Feedback
1	Most students completed generation and deployment within minutes. Times ranged from ~2 to ~20 min, with the majority under 10 min, compared to 45–60 min for manual development.
2	Reported difficulty was consistently low (1–3), with most students rating the process as “very easy.”
3	All students confirmed that the generated applications followed the expected layered architecture (Controller-Service-Repository).
4	All participants verified that home, list, and form pages were generated correctly.
5	Functionality was generally confirmed. Minor mismatches occurred in isolated cases due to route inconsistencies.
6	Students found the structure highly consistent with what was taught in class, which reinforced their understanding of architectural layering.
7	The tool reinforced understanding of Spring Boot architecture, averaging 4/5 on the Likert scale (range 3–5). Students noted it “made it simpler to remember what I learned,” “helped visualize how layers connect,” and clarified interactions between controllers, services, repositories, and views.
8	Ratings varied from 1 to 5. Some felt highly confident in extending the code, while others noted the initial volume of generated code was overwhelming.
9	Students reported learning about: (a) practical use of the terminal and Spring Boot JARs, (b) connecting controllers, services, and repositories, (c) working with validation, Swagger, and configuration files, and (d) appreciating the role of low-code tools in rapid prototyping. Overall, students recognized the tool’s value in reinforcing class concepts.

Beyond speed, students emphasized usability. They consistently rated the difficulty of launching the generated JAR file as very low, most often describing the process as ‘very

easy.’ This indicates that the generated applications were not only structurally correct but also straightforward to deploy.

All participants confirmed that the applications adhered to the expected layered architecture (Controller-Service-Repository) and included fully functional Thymeleaf pages (home, list, and form). Role-based access control was also generally successful, with only minor mismatches reported in isolated cases due to route inconsistencies.

From an educational perspective, students found the generated codebase highly consistent with classroom instruction, which reinforced their understanding of architectural layering. Several participants noted that XBoot made it simpler to remember key concepts, helped them visualize how layers connect, and clarified the interactions between controllers, services, repositories, and views.

Confidence in reusing or extending the generated applications was more mixed. Some students felt overwhelmed by the initial code volume, while others appreciated the modularity and maintainability of the generated structure, which they believed would facilitate future modifications.

In their open-ended feedback, students highlighted a variety of learning outcomes. They gained practical experience with terminal commands, CRUD functionality, RBAC, validation handling, Swagger documentation, and configuration management. Several also recognized the broader significance of low-code tools in accelerating prototyping and reducing boilerplate work, which provided them with a deeper appreciation of how such tools can complement traditional software engineering practice.

7.2.2. Discussion

The findings confirm that XBoot reliably transforms natural language requirements into runnable Spring Boot applications with minimal effort and strong consistency. The drastic reduction in time-to-application compared to manual coding suggests clear productivity benefits. Importantly, the automatically enforced architectural patterns and RBAC mechanisms reduced the likelihood of common structural and security flaws. For example, in prior coursework by the same cohort, approximately 60% of manually written Spring Boot projects exhibited issues such as missing role restrictions, incorrect layer mappings, or hardcoded credentials. By contrast, no XBoot-generated project displayed such flaws, underscoring its ability to encode best practices directly into the DSL.

From an educational perspective, the tool effectively reinforced architectural principles. Students confirmed that the generated layered structure mirrored classroom instruction and provided a clearer visualization of the interaction between components. The positive average on Q7, alongside qualitative comments, indicates that XBoot not only accelerated development but also served as a pedagogical scaffold for understanding layered design and separation of concerns.

At the same time, the mixed confidence levels in extending the generated code highlight an area for refinement. While the modular architecture facilitated reusability for some, others felt overwhelmed by the volume of boilerplate produced. This points to the need for additional scaffolding tools such as documentation, navigation aids, or partial code explanations to ease onboarding for less experienced learners.

Overall, the POC suggests that XBoot achieves both functional correctness and pedagogical value. It accelerates development, enforces best practices, and reinforces key educational outcomes. However, larger-scale and more controlled studies are needed to validate these preliminary insights. Future evaluations should also incorporate additional metrics, such as lines of code saved compared to manual development and the degree of editing required when DSL specifications are generated by LLMs.

7.3. XBoot Alingment with Common DSL Experiments Outcomes

Previous empirical studies on DSLs have reported consistent benefits in terms of developer effectiveness and efficiency compared to general-purpose languages (GPLs). For example, Voelter and Visser [2] showed that DSLs can improve comprehension by providing higher-level abstractions, while Chavarriaga et al. [6] and da Rocha et al. [7] demonstrated that XML-based DSLs can streamline development in specialized domains. More recent work has emphasized developer experience, where low-code and DSL-based solutions improve productivity and reduce configuration overhead compared to traditional GPL development [5,30,34].

Our proof-of-concept results with XBoot are consistent with these findings. Students were able to generate fully functional, structurally correct Spring Boot applications in minutes, whereas similar applications developed manually in prior coursework required 45–60 min. Structural and security-related flaws, which were common in manually coded projects, were absent in the XBoot-generated systems. These outcomes mirror prior DSL experiments in demonstrating gains in correctness and efficiency, while also extending the discussion to educational impact. In particular, participants reported that XBoot reinforced their understanding of layered architecture, role-based access control, and validation.

Unlike industrial DSL evaluations that mainly emphasize productivity in professional contexts, our results highlight how DSL-driven modeling can scaffold novice developers. This aligns with the broader DSL literature [2,30], while contributing a novel perspective on the pedagogical value of DSLs in backend software engineering education.

7.4. Limitations and Observations

It is important to contextualize the findings within the nature of this POC evaluation. The study involved a small and homogeneous cohort of 10 senior undergraduate students, which means the results should be interpreted as exploratory rather than statistically generalizable. Some participants conducted the experiment remotely while engaged in cooperative training, and no formal instruction on XBoot's DSL syntax was provided. Students relied solely on XBoot GPT to convert natural language descriptions into valid DSL specifications. While this reflects realistic usage of AI-assisted modeling in modern low-code environments, it introduced a degree of inconsistency, especially in datatype declarations and relationship formats, that could have been mitigated with targeted DSL instruction.

While the majority of student participants successfully generated and executed their applications using XBoot without requiring manual intervention, several limitations emerged that were primarily rooted in the intermediate DSL specifications produced via the GPT assistant, rather than in the XBoot generator itself.

However, a few participants reported issues with RBAC. Upon inspection, these were traced to mismatches between DSL-defined routes and controller paths. Two common issues were observed:

- **Route Mismatch:** An example of a route mismatch in the DSL is shown in Box 13.:

Box 13. DSL Snippet Illustrating a Route Mismatch.

```
<Route path="/enrollments/create" view="enrollment-create" method="POST"
controller="EnrollmentController" role="ADMIN"/>
```

This mismatch resulted in the generated Spring Security rule shown in Box 14.

Box 14. Generated Spring Security Rule from Mismatched Route.

```
.requestMatchers("/enrollments/create").hasRole("ADMIN")
```

However, the corresponding form was accessed via “/enrollments/form”, resulting in the page being unprotected. The issue was resolved by adjusting the rule as shown in Box 15.

Box 15. Corrected Spring Security Rule Protecting the Enrollment Form.

```
.requestMatchers("/enrollments/form").hasRole("ADMIN")
```

- Path Naming: Another case of mismatch arose from singular vs. plural path naming, where the controller was mapped as shown in Box 16.

Box 16. REST Controller Mapping with Plural Path (/enrollments).

```
@RequestMapping("/enrollments")
```

As with the previous issue, this was resolved by updating the route definition to ensure consistency.

These cases highlight the importance of maintaining strict alignment between DSL routes, controller mappings, and security configurations. To address this, XBoot GPT was enhanced with smarter path suggestions and stricter validation logic to reduce the likelihood of such mismatches in future use.

In addition, some students reported other DSL-related issues via informal communication channels such as WhatsApp. These included:

- **Incorrect Data Types:** A common mistake involved specifying primitive types (e.g., Long) in relationship fields, when a class reference (e.g., Course, Student) was required. These led to invalid JPA mappings and compilation errors. Students resolved the issue by correcting the field types, and XBoot GPT was updated to enforce entity references more strictly in generated DSL code.
- **Improper Relationship Formatting:** In multiple instances, relationships were defined using lowercase, hyphenated values such as “many-to-one” instead of the expected “ManyToOne” format. Due to the case-sensitive nature of the XBoot schema, these entries were either ignored or incorrectly parsed, leading to missing or incomplete relationship mappings. This prompted an update to XBoot GPT to generate only schema-compliant relationship types.
- **Advanced Customization Error—Date Field:** One student experimented by changing a flight departure field’s type from String to Date. While the XML was valid, the generated Java class lacked the required import (import java.util.Date). This caused a compilation error. After manually adding the required library, the application compiled but failed at runtime due to Thymeleaf expecting a String rather than a Date type. The student resolved this by adjusting the controller, service, and Thymeleaf template to support a date picker input. Although this required additional effort, it demonstrated that XBoot-generated code is modular and extensible for more advanced development scenarios.

These issues also exposed a broader design limitation: XBoot currently defaults temporal fields to String to reduce dependency complexity and facilitate smoother onboarding. While suitable for most beginners, this limits type expressiveness. Future releases will support optional handling for Java temporal types (Date, LocalDate, LocalDateTime), with automatic imports and Thymeleaf-compatible view rendering.

Despite these issues, students were generally able to debug and fix problems with minimal effort. When compilation errors occurred, XBoot-generated projects made it easy to locate the issue due to standard naming and file structure. Students used IDE error messages to trace and correct faulty XML specifications or Java code lines. This highlights the modularity and transparency of the generated artifacts.

Overall, these observations are consistent with the POC nature of the study: they demonstrate that the tool works as intended and can provide provisional learning benefits, while also highlighting the need for larger, more diverse cohorts and controlled settings in future evaluations.

7.5. Threats to Validity

While the evaluation of XBoot demonstrates promising results in terms of generation fidelity, student productivity, and educational value, it is important to acknowledge potential threats to the validity of these findings. This section discusses key internal, external, construct, and conclusion validity concerns.

7.5.1. Internal Validity

The internal validity of the experiment may be influenced by individual differences among participants. Although all students had prior exposure to Java and Spring Boot, variations in programming experience, problem-solving skills, and familiarity with XML may have affected their performance. Furthermore, students relied on XBoot GPT to generate the DSL specifications, and any inaccuracies in the GPT-generated output may have introduced confounding variables unrelated to XBoot itself. For example, some DSL errors were caused by incorrect relationship type formats or invalid data types, which required manual intervention by the participants.

7.5.2. External Validity

The study was conducted with a relatively small and homogeneous group of ten senior undergraduate students from a single institution. While the results suggest that XBoot can support backend development in educational settings, these findings may not generalize to novice programmers, non-technical users, or students from other academic backgrounds. Additionally, all participants had been previously trained in Spring Boot, which may have influenced their ability to recognize and correct DSL issues. The remote nature of the experiment, conducted during students' cooperative training, may also differ from in-class use scenarios where peer support and instructor guidance are more readily available. Overall, the small sample size and the homogeneity of the group mean that the findings should be interpreted only as an exploratory pilot, with larger-scale and more diverse studies required to draw general conclusions.

7.5.3. Construct Validity

The constructions used to evaluate productivity and learning, such as time-to-completion, self-reported understanding, and confidence, may not fully capture the depth of architectural comprehension or long-term learning outcomes. While these measures provide insight into usability and perceived clarity, they are inherently subjective. The evaluation did not include objective tests of students' retained knowledge or ability to generalize beyond the generated codebase. Additionally, the use of DSL modeling as

a proxy for backend design skill assumes that learners understand the DSL semantics, which was not directly tested.

7.5.4. Conclusion Validity

Given the limited sample size (four participants) and exploratory nature of the evaluation, it is not possible to make statistically significant claims about the effectiveness of XBoot and exploratory nature of the evaluation, it is not possible to make statistically significant claims about the effectiveness of XBoot. The findings suggest clear trends in reduced development time and high user satisfaction, but larger-scale studies are needed to validate these conclusions across broader populations. Furthermore, while students were able to generate and deploy applications quickly, the need for occasional DSL corrections (as discussed in Section 7.3) suggests that the overall automation pipeline still relies partially on student judgment and technical intuition.

7.5.5. Mitigation and Future Work

To address these threats, future evaluations of XBoot will involve larger cohorts, randomized controlled designs, and direct comparison with manual development workflows. Planned in-class studies will compare student performance under three conditions: manual coding, DSL-based modeling, and natural language-driven GPT modeling. Additionally, integrating in-tool validation and guided DSL editors may help reduce errors and improve reliability for novice users.

8. XBoot in Education

In addition to rapid prototyping, XBoot was designed to support backend software engineering education. Drawing on the authors' experience teaching undergraduate web engineering, it addresses recurring challenges students face when learning Spring Boot.

The observations are based on third-year software engineering students who, despite completing multiple Java-focused courses (OOP, data structures, design patterns), struggled with Spring Boot's framework-specific requirements. Common issues included:

- Project setup errors: Misplaced `@SpringBootApplication` classes or incorrect package hierarchies broke component scanning. Students also confused REST and view controllers, often implementing one but not both.
- Thymeleaf difficulties: Form bindings and constructs such as `th:each` were frequently mishandled. Missing dependencies in lab setups (e.g., omitting Spring Web) caused build failures and wasted debugging time.
- Security gaps: Students implemented login but often failed to enforce RBAC in final projects.

These challenges showed that novices struggled more with Spring Boot's structure and configuration than with application logic, slowing progress and limiting architectural focus.

XBoot mitigates these difficulties by shifting effort from manual coding to high-level modeling. With a concise XML DSL, students specify entities, relationships, routes, and roles, and the generator produces:

- A valid Maven project with correct package hierarchy and entry point.
- REST and view controllers with consistent routing.
- Thymeleaf templates for forms and listings with valid bindings.
- Spring Security enforcing RBAC at both view and API layers.
- All required dependencies included automatically.

For example, in the StudentCourses case study, a 40-line DSL produced over 900 lines of Java and HTML, scaffolding the entire backend stack. The result is a complete, functional

application that reflects best practices in separation of concerns, layered architecture, and security, providing students with a template to study, run, and extend.

8.1. Educational Impact and Instructional Advantages of XBoot

The following discussion highlights the observed classroom benefits for students, practical advantages for instructors, and potential curriculum positioning of XBoot.

Classroom Benefits and Observations

Although XBoot has not yet been subjected to a formal educational study, its use in labs, quizzes, exams, and projects has revealed several benefits:

- Students complete backend assignments more quickly and with greater confidence.
- Structural and configuration errors, particularly in project setup, are reduced.
- REST and view controller roles become clearer through consistent scaffolding.
- Thymeleaf integration is less error-prone due to ready-made templates.
- Role-based authorization is easier to apply using declarative modeling rather than manual annotations.

By starting from high-level models and incrementally exploring the generated code, students face a lower entry barrier and are encouraged to reason architecturally from the outset.

Instructor Utility

XBoot also offers substantial advantages for instructors.

- Enables the design of structured, multi-stage assignments.
- Provides a clean codebase to demonstrate layered design, secure routing, and service abstraction.
- Minimizes time spent on setup errors, allowing more time for teaching architecture and design thinking.
- Facilitates model-to-code traceability, making it easier to explain how changes in design are reflected in implementation.

Positioning in the Curriculum

XBoot fits well within backend development courses in software engineering, information systems and computer science. It also supports broader themes, such as:

- Web engineering.
- Secure application development.
- Model-driven engineering.
- Low-code and DSLs.
- Component-based system design.

In summary, XBoot helps students and instructors move beyond boilerplate and misconfiguration by providing a structured, declarative, and secure approach to backend development. It bridges the gap between theory, modeling, and implementation, offering a strong foundation for full-stack learning in academic settings.

8.2. Pedagogical Benefits: Structured Learning of Spring Boot and Layered Architecture

A recurring challenge in backend software engineering education is ensuring that students not only build functional applications but also internalize sound architectural practices. In manual Spring Boot projects, novices often bypass the service layer or embed business logic in controllers, undermining modularity and maintainability.

XBoot addresses this challenge by generating applications that consistently follow a layered model with entities, repositories, services, REST and view controllers, and Thymeleaf templates. This scaffolding provides tangible examples of best practices, guiding students toward modular design aligned with established conventions.

XBoot is not a replacement for learning Spring Boot fundamentals but a structured scaffold that lowers the entry barrier. By abstracting setup, dependency management, and annotation complexity, it enables students to focus on architectural concerns such as request handling, validation, routing, and access control. As familiarity grows, learners can reconstruct or extend these layers manually.

A potential risk is over-reliance on generated artifacts, which may limit coding practice. To mitigate this, phased assignments are recommended: students can begin with DSL-based generation, then progressively modify service logic, extend validation, or add secured endpoints. This approach balances automation with hands-on experience, reinforcing both architectural understanding and coding competence.

9. XBoot in the Context of AI-Driven Development Tools

The rise in AI coding assistants such as GitHub Copilot, Amazon CodeWhisperer, and GPT-based generators has introduced new paradigms in software development. These tools translate natural language prompts into code, enabling rapid prototyping but often at the expense of traceability, consistency, maintainability, and educational suitability [37].

XBoot addresses these shortcomings through a model-driven approach grounded in a formal XML-based DSL. Unlike probabilistic outputs from AI, XBoot delivers deterministic, traceable, and security-aware code generation. Its distinct advantages can be summarized across five areas:

1. **Deterministic generation:** AI outputs vary with phrasing, while XBoot ensures repeatable, auditable results from the same XML input.
2. **Design-to-code transparency:** Each DSL tag maps directly to entities, controllers, views, or security rules, making implementation artifacts traceable to requirements.
3. **Enforced layering:** Whereas AI may blur responsibilities, XBoot consistently applies MVC principles, producing separate entities, repositories, services, and controllers.
4. **Integrated security:** Role attributes in `<Route>` and `<Connector>` tags are translated into Spring Security rules and Thymeleaf restrictions, ensuring consistent RBAC enforcement.
5. **Structured refinement:** XBoot supports incremental evolution of models through schema validation, avoiding the instability of re-prompting AI tools.

Rather than replacing AI, XBoot complements it. GPT-based tools can draft DSL snippets from informal descriptions, after which XBoot validates, structures, and generates a complete application. In this way, AI provides flexibility for exploration, while XBoot guarantees transparency, architectural discipline, and security. Together, they support both rapid prototyping and structured learning, making XBoot well suited for academic and professional contexts.

10. Trade-Offs of Learning with XBoot vs. Manual Development

XBoot lowers the entry barrier to backend development by allowing students to model and generate complete Spring Boot applications from compact specifications. This enables learners to focus on architectural concepts such as layered design, modularity, and role-based access control without being hindered by configuration overhead. By scaffolding fully functional applications, it provides a clear entry point for understanding backend architecture.

However, this abstraction introduces trade-offs. Students risk gaining only a superficial grasp of Spring Boot internals, such as how annotations (`@Service`, `@PreAuthorize`, `@RequestMapping`), dependency injection, or request routing shape application behavior. Overreliance on generated code can create “black box” learning, limiting skills in debugging, configuration, and manual setup competencies essential in professional contexts.

To mitigate this, XBoot should be used as a complement rather than a substitute for manual development. Educators can start with DSL-based modeling to emphasize architecture and security, then progressively shift to coding tasks where students adapt or rewrite generated artifacts. This staged approach helps learners first understand *what* needs to be built before mastering how to build it.

XBoot GPT further supports this path by transforming natural language descriptions into valid DSL specifications, accelerating design and allowing instructors to focus on patterns and principles rather than syntax. Together, XBoot and its assistant foster a balanced strategy: rapid early comprehension and architectural reasoning paired with the rigor of traditional Spring Boot development, ensuring students gain both high-level design thinking and hands-on implementation skills.

11. Conclusions and Future Work

This study introduced XBoot, a lightweight model-driven tool for generating secure and modular Spring Boot applications from XML specifications. Designed with education in mind, XBoot enables students and instructors to focus on architecture, entity relationships, and access control rather than framework configuration and boilerplate coding.

By adopting a component-connector model and embedding role-based access control at the specification layer, XBoot produces maintainable and traceable backend systems aligned with established engineering principles. The generated code spans all backend layers, with security consistently enforced across REST endpoints and user interfaces.

The StudentCourses case study demonstrated that a compact DSL file can generate a complete system including entities, repositories, services, controllers, security settings, and Thymeleaf views with minimal effort. Author-driven validation and a structured student experiment confirmed the correctness, usability, and educational value of the generated applications.

The evaluation results highlight several advantages. XBoot reduced development time dramatically (from ~45–60 min manually to ~5 min using DSL specifications), eliminated common errors in role restrictions and architectural layering, and produced applications directly aligned with classroom instruction. Student feedback further confirmed its usability and effectiveness as an educational tool. Together, these findings demonstrate XBoot's potential to accelerate learning while reinforcing best practices in secure backend development.

Future work will extend frontend security enforcement. While unauthorized requests are already blocked at the controller level through Spring Security annotations (e.g., `@PreAuthorize`), current view templates do not dynamically reflect user permissions. Upcoming versions will incorporate Thymeleaf's `sec:authorize` attributes into generated views, ensuring that navigation links and form elements are visible only to users with appropriate roles. This will bring frontend and backend security layers into full alignment and further enhance the user experience. In addition, benchmarking XBoot against alternative frameworks on equivalent case studies is planned to provide a broader comparative evaluation.

12. Current Limitations of XBoot

While XBoot shows strong educational value and practical usability, several limitations remain. First, its security model relies on statically defined roles in the DSL, limiting support for dynamic scenarios such as runtime role assignment, tenant-based authorization, or integration with external identity providers. This reduces suitability for enterprise-grade applications requiring fine-grained access control.

Second, view generation is restricted to basic data entry and listing. Advanced UI logic, client-side validation, and responsive design must still be implemented manually, constraining its use for complex or production-ready interfaces.

Third, XBoot is tightly coupled with Spring Boot and a fixed project structure, which may limit adaptability to alternative frameworks or modular architectures. Although the DSL enforces architectural separation, it can also abstract details that advanced learners or professionals may wish to study more directly.

Finally, while promising for instruction and prototyping, XBoot has not yet been evaluated in large-scale classroom trials, leaving its broader educational impact to be assessed in future empirical studies.

Author Contributions: Conceptualization, B.Y.A. and M.K.N.; methodology, B.Y.A. and M.K.N.; software, B.Y.A.; validation, B.Y.A. and M.K.N.; formal analysis, B.Y.A.; investigation, M.K.N.; resources, M.K.N.; data curation, M.K.N.; writing—original draft preparation, B.Y.A.; writing—review and editing, M.K.N.; visualization, M.K.N.; supervision, B.Y.A.; project administration, B.Y.A.; funding acquisition, B.Y.A. and M.K.N. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Data Availability Statement: All data are provided in the text of this paper. The XBoot framework binary and usage instructions are available publicly on GitHub [38].

Acknowledgments: The authors acknowledge the use of an AI-based tool (PaperPal) to assist in proofreading and improving the clarity of the English language in this manuscript. The tool was not used for generating original content, analysis, or results. All substantive ideas, experiments, and conclusions are the authors' own.

Conflicts of Interest: The authors declare no conflicts of interest in the publication of this paper.

Appendix A

Appendix A.1 XBoot Execution Checklist Generated at Console

```

✓ XML is valid.
✓ Entity class generated: Flight.java
✓ Repository generated: FlightRepository.java
✓ Service generated: FlightService.java
✓ error.html view generated.
✓ Thymeleaf views generated for: Flight
✓ View controller generated: FlightViewController.java
✓ HomeController generated.
✓ AppErrorController generated.
✓ Entity class generated: Passenger.java
✓ Repository generated: PassengerRepository.java
✓ Service generated: PassengerService.java
✓ error.html view generated.
✓ Thymeleaf views generated for: Passenger
✓ View controller generated: PassengerViewController.java
✓ HomeController generated.
✓ AppErrorController generated.
✓ Entity class generated: Reservation.java
✓ Repository generated: ReservationRepository.java
✓ Service generated: ReservationService.java
✓ error.html view generated.
✓ Thymeleaf views generated for: Reservation
✓ View controller generated: ReservationViewController.java
✓ HomeController generated.
✓ AppErrorController generated.
✓ REST Controller generated with Swagger annotations: FlightController.java
✓ RestExceptionHandler generated.
✓ REST Controller generated with Swagger annotations: PassengerController.java
✓ RestExceptionHandler generated.
✓ REST Controller generated with Swagger annotations: ReservationController.java
✓ RestExceptionHandler generated.
i RouteBasedViewController generation skipped — now handled by entity-specific controllers.
✓ HomeController.java generated.
✓ SecurityConfig.java generated using Spring Security defaults (Option 1).
✓ UserDetailsServiceConfig.java generated with external configuration support.
✓ MainApp.java generated.
✓ pom.xml generated with Thymeleaf Spring Security extras.
✓ application.properties generated with DB, logging, and security config.
✓ WebConfig generated for method override support.
✓ SwaggerConfig.java generated.
✓ messages.properties file generated.
✓ XBoot full backend code generated for: FlightBookingApp
✓ Code generation completed for application: FlightBookingApp

```

Appendix A.2 XBoot EBNF Representation

The core grammar of the XBoot DSL is represented below using the Extended Backus-Naur Form (EBNF). This defines the structure of the valid XBoot XML specifications.

```

Application ::= '<Application name=' Name "'>'
              (Entity | Controller | Service | Repository | Route | Connector | View)+
              '</Application>'

Entity       ::= '<Entity name=' Name "'>' Field+ '</Entity>'

Field        ::= '<Field name=' Name "' type=' Type "' Required? Relation? '>'
              Validation*
              '</Field>'

Required     ::= 'required="true"' | 'required="false"'

Relation     ::= 'relation=' RelationType '"'

RelationType ::= 'OneToOne' | 'OneToMany' | 'ManyToOne' | 'ManyToMany'

Validation   ::= '<Validation type=' ValidationType "' ValidationAttr* '/>'
ValidationType ::= 'NotBlank' | 'NotNull' | 'Size' | 'Pattern' | 'Email'
ValidationAttr ::= 'min=' Int "' | 'max=' Int "' | 'regex=' Regex '"'

Service      ::= '<Service name=' Name "'>'

Repository   ::= '<Repository name=' Name "'>'

Controller   ::= '<Controller name=' Name "' for=' Name "'>'

Route        ::= '<Route path=' Path "' method=' Method "'
              ViewAttr? ControllerAttr? RoleAttr? '/>'
View         ::= '<View for=' Name "' template=' TemplateName "'>'

ConnectorType ::= 'REST' | 'MVC' | 'async' | 'secured'
Method        ::= 'GET' | 'POST' | 'PUT' | 'DELETE'

TemplateName ::= Name ('.' Name)* ('.' 'html' )?

RoleList      ::= Role ('.' Role)*
Role          ::= 'USER' | 'ADMIN' | Name

Name          ::= Letter (Letter | Digit)*
Type          ::= 'String' | 'Integer' | 'Double' | 'Boolean' | 'Date' | Name
Path          ::= '/' Name ('/' Name)*

Int           ::= Digit+
Regex         ::= Java-compatible-regex

Letter        ::= 'a'..'z' | 'A'..'Z'
Digit         ::= '0'..'9'

```

Appendix A.3 Generated Security Configuration (Excerpt)

Below is an excerpt of the generated Spring Security configuration file, automatically produced by XBoot for the StudentCourses system.

```

@Configuration

public class SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {

        http

            .authorizeHttpRequests(auth -> auth

                .requestMatchers("/home", "/css/**", "/js/**", "/images/**").permitAll()

                .requestMatchers("/student/**").hasRole("USER")

                .requestMatchers("/course/**").hasRole("USER")

                .requestMatchers("/enrollment/**").hasRole("USER")

                .anyRequest().authenticated()

            )

            .formLogin(form -> form

```

Appendix A.4 Students–Courses Case Study DSL Description

```

<Application name="StudentCourses">
  <!-- Entities -->
  <Entity name="Student">
    <Field name="name" type="String" required="true">
      <Validation type="NotBlank"/>
      <Validation type="Size" min="2" max="50"/>
    </Field>
    <Field name="email" type="String">
      <Validation type="Email"/>
    </Field>
    <Field name="mobile" type="String">
      <Validation type="Pattern" regex="\d{10}"/>
    </Field>
  </Entity>

  <Entity name="Course">
    <Field name="title" type="String" required="true">
      <Validation type="NotNull"/>
    </Field>
    <Field name="description" type="String"/>
  </Entity>

  <Entity name="Enrollment">
    <Field name="student" type="Student" relation="ManyToOne" required="true"/>
    <Field name="course" type="Course" relation="ManyToOne" required="true"/>
    <Field name="grade" type="String"/>
  </Entity>

  <!-- Controllers -->
  <Controller name="StudentController" for="Student"/>
  <Controller name="CourseController" for="Course"/>
  <Controller name="EnrollmentController" for="Enrollment"/>

  <!-- Routes -->
  <Route path="/home" view="home" method="GET" role="USER"/>
  <Route path="/students" view="student-list" method="GET" controller="StudentController" role="USER"/>
  <Route path="/courses" view="course-list" method="GET" controller="CourseController" role="USER"/>
  <Route path="/enrollments" view="enrollment-list" method="GET" controller="EnrollmentController" role="USER"/>
  <Route path="/enrollments/new" view="enrollment-form" method="POST" controller="EnrollmentController" role="ADMIN"/>

  <!-- Security Connectors -->
  <Connector from="StudentController" to="Student" role="USER" type="REST"/>
  <Connector from="CourseController" to="Course" role="USER" type="REST"/>
  <Connector from="EnrollmentController" to="Enrollment" role="USER" type="REST"/>

</Application>

```

Appendix A.5 FlightsBooking XBoot XML-DSL Description

```

<Application name="flightbooking">
  <Entity name="Passenger">
    <Field name="passengerId" type="Long" required="true"/>
    <Field name="passengerName" type="String">
      <Validation type="NotBlank"/>
    </Field>
    <Field name="passengerNationality" type="String">
      <Validation type="NotBlank"/>
    </Field>
    <Field name="passengerAge" type="Integer" required="true">
      <Validation type="Min" value="0"/>
    </Field>
    <Field name="passengerPhone" type="String">
      <Validation type="Pattern" regex="\d{10,15}"/>
    </Field>
    <Field name="passengerEmail" type="String">
      <Validation type="Email"/>
    </Field>
  </Entity>

  <Entity name="Airplane">
    <Field name="airplaneId" type="Long" required="true"/>
    <Field name="airplaneType" type="String">
      <Validation type="NotBlank"/>
    </Field>
    <Field name="airplaneSeatsCount" type="Integer" required="true">
      <Validation type="Min" value="1"/>
    </Field>
  </Entity>

  <Entity name="Flight">
    <Field name="flightNumber" type="String" required="true">
      <Validation type="NotBlank"/>
    </Field>
    <Field name="flightOrigin" type="String">
      <Validation type="NotBlank"/>
    </Field>
    <Field name="flightDepartureTime" type="String">
      <Validation type="NotBlank"/>
    </Field>
    <Field name="flightDuration" type="Integer" required="true">
      <Validation type="Min" value="1"/>
    </Field>
  </Entity>

  <Entity name="Reservation">
    <Field name="flightNumber" type="Flight" relation="ManyToOne">
      <Validation type="NotBlank"/>
    </Field>
    <Field name="passengerId" type="Passenger" relation="ManyToOne" required="true"/>
    <Field name="airplaneId" type="Airplane" relation="ManyToOne" required="true"/>
    <Field name="seatNumber" type="String"/>
    <Field name="status" type="String"/>
  </Entity>

  <Controller name="PassengerController" for="Passenger"/>
  <Controller name="AirplaneController" for="Airplane"/>
  <Controller name="FlightController" for="Flight"/>
  <Controller name="ReservationController" for="Reservation"/>
  <Route path="/passengers" view="passenger-list" method="GET" controller="PassengerController" role="USER"/>
  <Route path="/airplanes" view="airplane-list" method="GET" controller="AirplaneController" role="USER"/>
  <Route path="/flights" view="flight-list" method="GET" controller="FlightController" role="USER"/>
  <Route path="/reservations" view="reservation-list" method="GET" controller="ReservationController" role="USER"/>
  <Route path="/reservations/new" view="reservation-form" method="POST" controller="ReservationController" role="ADMIN"/>
  <Connector from="PassengerController" to="Passenger" role="USER" type="REST"/>
  <Connector from="AirplaneController" to="Airplane" role="USER" type="REST"/>
  <Connector from="FlightController" to="Flight" role="USER" type="REST"/>
  <Connector from="ReservationController" to="Reservation" role="USER" type="REST"/>

</Application>

```

Appendix A.6 Student Experiment Manual

This appendix reproduces the manual provided to student participants during the XBoot evaluation. It outlines the experiment procedure, tool usage instructions, and the formal case study descriptions used in the study.

XBoot Student Experiment Manual

This manual guides you through the controlled experiment using the XBoot tool for low-code generation of Spring Boot full-stack applications. You will complete two case studies and provide structured feedback.

Folder Structure

- You will receive a zip file containing a folder named xboot/with:
- xboot.jar—the executable generator
- input/— place your XML DSL file here (xboot.xml)
- output/— generated project will appear here
- schema/— contains XBootSchema.xsd used for validation

What You Will Do

You will complete two tasks:

1. Generate the StudentCourses application
2. Generate the FlightBooking application

For each task:

- Describe the application to the XBoot GPT assistant in natural language
- Save the XML DSL to input/xboot.xml
- Run the JAR file
- Open the generated project in your IDE (e.g., Eclipse or IntelliJ)
- Explore the structure and test the views
- Complete the feedback form

Instructions

1. Access the GPT assistant: ChatGPT—XBoot DSL Generator
2. Provide a natural language prompt (see example below): *“Create an application for employees and departments. Employees have name, email, salary. A department can hold many employees. . .”*
3. Adapt this for StudentCourses or FlightBooking.
4. Copy the XML to input/xboot.xml.
5. Run the tool via:

`java -jar xboot.jar input/xboot.xml schema/XBootSchema.xsd`, Or double-click the .jar file.

6. Open the project in your IDE.
7. Navigate and test generated views.
8. Fill out the feedback form.
9. Repeat the same for the second case study.

Submission

Submit the following:

- Feedback form
- Screenshots of the running applications
- Two DSL XML files
- Generated output projects (as zip)

Testing URLs

StudentCourses:

- /home

- /students
- /courses
- /enrollments
- /enrollments/new (*Admin only*)

FlightBooking:

- /flights
- /passengers
- /reservations
- /reservations/new (*Admin only*)

Check application.properties for username/password.

Case Study Descriptions

Case Study 1: StudentCourses

- Entities:
 - Student: name, email, mobile
 - Course: title, description
 - Enrollment: student (ManyToOne → Student), course (ManyToOne → Course), grade
- Relationships:
 - Student enrolls in many courses via Enrollment
 - Enrollment links Student and Course
- Requirements:
 - Use @NotBlank, @Email, @Pattern for validation
 - User views for listing and creating enrollments
 - Protect enrollment creation with ADMIN role

Case Study 2: FlightBooking

- Entities:
 - Flight: flightNumber, origin, destination, departureTime
 - Passenger: name, passportNumber, email
 - Reservation: flight (ManyToOne → Flight), passenger (ManyToOne → Passenger), seatNumber, status
- Relationships:
 - One passenger and one flight per reservation
 - A flight/passenger can have many reservations
- Requirements:
 - Use validation annotations
 - List/form views for reservations
 - Protect reservation form with ADMIN role

Appendix A.7 Student Feedback Form

This form must be filled out after completing each case study using the XBoot tool and GPT assistant. Please answer all sections carefully. Your feedback supports research on low-code Spring Boot prototyping.

Case Studies Metrics:

A. Timing Metrics

1. Time (in minutes) from natural language description to working application:
2. How difficult was it to run the JAR and launch the app? (1 = Very Easy, 5 = Very Difficult).

- B. Structural Correctness
1. Was a layered architecture (Controller–Service–Repository) generated? (Yes/No/Partial):

2. Were Thymeleaf view pages (home, list, form) generated? (Yes/No):

3. Was role-based access control working as expected? (Yes/No):

4. Was the structure similar to what you learned in class? If no, explain:
- C. Learning and Confidence
1. Did this tool help reinforce your understanding of Spring Boot architecture? (1–5):

2. Do you feel confident reusing or extending the generated app? (1–5):

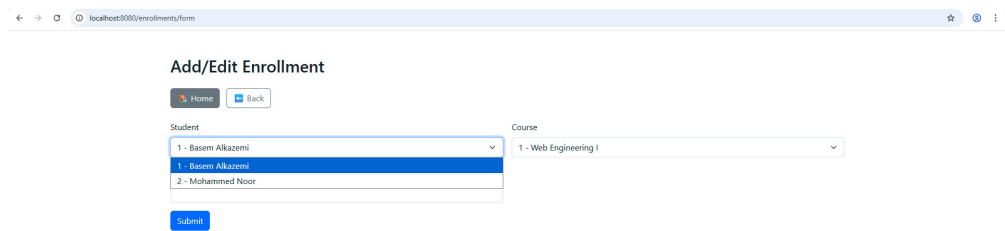
3. What did you learn from this experience?

Appendix B

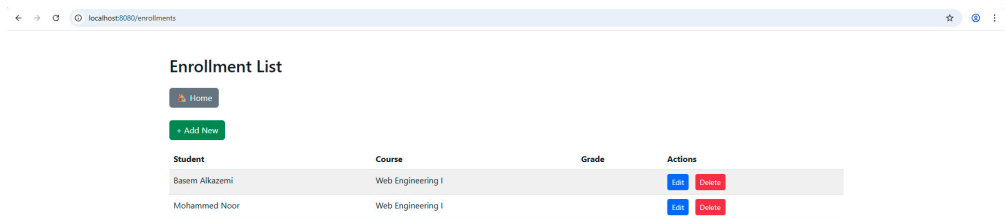
Appendix B.1 Courses Listing Page



Appendix B.2 Students' Enrollment Form



Appendix B.3 Students' Enrollments in Courses Listing Page



References

1.

Walls, C. *Spring Boot in Action*, 6th ed.; Manning Publications: Shelter Island, NY, USA, 2023.

2.

Voelter, M.; Visser, E. DSL Engineering: Designing, Implementing and Using Domain-Specific Languages. In *Software Language Engineering*; Springer: Berlin/Heidelberg, Germany, 2011; pp. 24–44.

3.

Bucaioni, A.; Cicchetti, A.; Ciccuzzi, F. Modelling in low-code development: A multivocal systematic review. *Softw. Syst. Model.* **2022**, *21*, 1959–1981. [\[CrossRef\]](#)

4.

Soldani, J.; Tamburri, D.A.; Van Den Heuvel, W.-J. The pains and gains of microservices: A systematic grey literature review. *J. Syst. Softw.* **2018**, *146*, 215–232. [\[CrossRef\]](#)

5.

Guthardt, L.; Kosiol, J.; Hohlfeld, O. Lowcode vs. the developer: An empirical study on the developer experience and efficiency of a nocode platform. In *Proceedings of the ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems*, Linz, Austria, 22–27 September 2024. [\[CrossRef\]](#)

6. Chavarriaga, E.; Jurado, F.; Díez, F. An approach to build XML-based domain specific languages solutions for client-side web applications. *Comput. Lang Syst. Struct.* **2017**, *49*, 133–151. [\[CrossRef\]](#)
7. da Rocha, M.M.; Favela, J.; Muchaluat-Saade, D.C. Design and Evaluation of an XML-Based Language for Programming Socially Assistive Robots. *Int. J. Soc. Robot.* **2025**, *17*, 1305–1333. [\[CrossRef\]](#)
8. Alonso, S.; Kalinowski, M.; Ferreira, B.; Barbosa, S.D.J.; Lopes, H. A systematic mapping study and practitioner insights on the use of software engineering practices to develop MVPs. *Inf. Softw. Technol.* **2023**, *156*, 107144. [\[CrossRef\]](#)
9. Rädler, S.; Berardinelli, L.; Winter, K.; Rahimi, A. Bridging MDE and AI: A systematic review of domain-specific modeling for AI software systems. *Softw. Syst. Model.* **2024**, *24*, 445–469. [\[CrossRef\]](#)
10. Taylor, R.N.; Medvidovic, N.; Dashofy, E.M. Component and connector views in practice: An experience report. In Proceedings of the ACM International Conference on Model-Driven Engineering Languages and Systems (MODELS), Ann Arbor, MI, USA, 5–10 October 2017.
11. Fowler, M.; Lewis, J. Microservices: A Definition of This New Architectural Term. 2014. Available online: <https://martinfowler.com/articles/microservices.html> (accessed on 3 June 2025).
12. Newman, S.; Microservices, B.; Media, O. *Building Microservices: Designing Fine-Grained Systems*, 2nd ed.; O'Reilly Media: Sebastopol, CA, USA, 2021.
13. Dragoni, N.; Dustdar, S.; Larsen, S.; Mazzara, M. Microservices: Yesterday, Today, and Tomorrow. In *Present and Ulterior Software Engineering*; Mazzara, M., Meyer, B., Eds.; Springer: Cham, Switzerland, 2017; pp. 195–216. [\[CrossRef\]](#)
14. Johnsson, D.B.; Deogun, D.; Sawano, D.; Design, S.B. *Software and Service Security*; Manning Publications: Shelter Island, NY, USA, 2019.
15. Agnihotri, J.; Phalnikar, R. Development of performance testing suite using Apache JMeter. In *Advances in Intelligent Systems and Computing*; Springer: Singapore, 2018; pp. 317–326. [\[CrossRef\]](#)
16. Moreno-García, J.P.; García-Magariño, I.; Sánchez, C. A survey of the Selenium ecosystem. *Electronics* **2020**, *9*, 1067. [\[CrossRef\]](#)
17. Barakat, S.; Roque, E.B.; Sánchez, A.B.; Segura, S. Specification-Driven Code Generation for Interparameter Dependencies in Web APIs. In *Service-Oriented Computing—ICSOC 2022 Workshops (LNCS, vol. 13821)*; Springer: Cham, Switzerland, 2023; pp. 261–273. [\[CrossRef\]](#)
18. Mendix, Low-Code Application Development Platform. Available online: <https://www.mendix.com/platform/> (accessed on 25 May 2025).
19. Mew, L.; Field, D. A case study on using the Mendix low code platform to support a project management course. In Proceedings of the 2018 Information Systems & Computing Academic Professionals (ISCAP) Conference, Norfolk, VA, USA, 8–11 August 2018. Available online: <https://iscap.us/proceedings/2018/pdf/4621.pdf> (accessed on 17 April 2025), ISSN 2473–3857.
20. Bootify. Rapid Spring Boot Prototypes | Bootify.io. Available online: <https://bootify.io/> (accessed on 25 March 2025).
21. Pivotal Software, Inc. Spring Initializr. Available online: <https://start.spring.io/> (accessed on 11 November 2024).
22. Vuksan, B.; Jadhav, S.G.; McCarthy, J. *Spring Roo in Action*; Manning Publications: Shelter Island, NY, USA, 2012.
23. Project, O. OpenXava Reference Guide. Available online: <https://openxava.org/OpenXavaDoc/> (accessed on 12 January 2025).
24. Gómez, O.S.; Miranda, R.H.R.; Verdín, K.C. CRUDyLeaf: A DSL for Generating Spring Boot REST APIs from Entity CRUD Operations. *Cybern. Inf. Technol.* **2020**, *20*, 3–14. [\[CrossRef\]](#)
25. Guégain, E.; Bonvoisin, A.; Acher, M.; Quinton, C.; Rouvoy, R. On the Interplay between Performance Indicators in Configurable Software: The Jhipster Case Study. 2023. Available online: https://papers.ssrn.com/sol3/papers.cfm?abstract_id=4640065 (accessed on 14 June 2025).
26. Halin, A.; Nuttinck, A.; Acher, M.; Devroey, X.; Perrouin, G.; Baudry, B. Test Them All, Is It Worth It? Assessing Configuration Sampling on the JHipster Web Development Stack. *Empir. Softw. Eng.* **2019**, *24*, 674–717. [\[CrossRef\]](#)
27. Slifka, J.; Pergl, R. Application of Normalized Systems Theory to pure functional code to achieve sustainability of web front-end applications. *J. Comput. Lang.* **2025**, *85*, 101346. [\[CrossRef\]](#)
28. Martínez-Lasaca, F.; Díez, P.; Guerra, E.; de Lara, J. Dandelion: A scalable, cloud-based graphical language workbench for industrial low-code development. *J. Comput. Lang.* **2023**, *76*, 101217. [\[CrossRef\]](#)
29. Chavarriaga, E.; Jurado, F.; Rodríguez, F.D. An approach to build JSON-based Domain Specific Languages solutions for web applications. *J. Comput. Lang.* **2023**, *75*, 101203. [\[CrossRef\]](#)
30. Pinho, D.; Aguiar, A.; Amaral, V. What about the usability in low-code platforms? A systematic literature review. *J. Comput. Lang.* **2023**, *74*, 101185. [\[CrossRef\]](#)
31. Dalibor, M.; Heithoff, M.; Michael, J.; Netz, L.; Pfeiffer, J.; Rumpe, B.; Varga, S.; Wortmann, A. Generating customized low-code development platforms for digital twins. *J. Comput. Lang.* **2022**, *70*, 101117. [\[CrossRef\]](#)
32. ISO/IEC 14977; Information Technology—Syntactic Metalanguage—Extended BNF (EBNF). ISO: Geneva, Switzerland, 1996.
33. Alkazemi, B.; Nour, M. XBoot DSL Generator. Available online: <https://tinyurl.com/2j3ejb78> (accessed on 18 July 2025).
34. Hagel, N.; Hili, N.; Bartel, A.; Koziolok, A. Turning Low-Code Development Platforms into True No-Code with Extensible DSLs. In Proceedings of the ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems, Linz, Austria, 22–27 September 2024. [\[CrossRef\]](#)

35. Shin, M.E.; Kang, T.; Gomaa, H. Secure Software Architectural Patterns Designed with Secure Connectors. In Proceedings of the 17th International Conference on Software Technologies ICSOFT, Lisbon, Portugal, 11–13 July 2022; pp. 484–491. [[CrossRef](#)]
36. Shin, M.E.; Gomaa, H.; Pathirage, D. A Software Product Line Approach for Feature Modeling and Design of Secure Connectors. In Proceedings of the 13th International Conference on Software Technologies, Porto, Portugal, 26–28 July 2018; pp. 506–517. [[CrossRef](#)]
37. Zhang, H.; Zhang, K.; Li, Z.; Li, J.; Li, J.; Li, Y.; Zhao, Y.; Zhu, Y.; Liu, F.; Li, G.; et al. Deep learning for code generation: A survey. *Sci. China Inf. Sci.* **2024**, *67*, 191101. [[CrossRef](#)]
38. Alkazemi, B.; Nour, M. XBoot: A Low-Code Generator for Spring Boot. Available online: <https://github.com/mkmmma/XBoot> (accessed on 9 June 2025).

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.